

DEPARTMENT OF ELECTRONICS & COMMUNICATION ENGINEERING

B.E. · Academic Year 2026-27



# Analysis, Simplification and Arduino-Based Implementation

**GATE EC Logic Circuit**

 Documentation

Submitted by **Samiksha S**  
Subject **Digital Electronics & Logic Design**  
Date **June 5, 2026**



# Table of Contents

---

## CONTENTS

---

|           |                                            |          |
|-----------|--------------------------------------------|----------|
| <b>1</b>  | <b>Abstract</b>                            | <b>2</b> |
| <b>2</b>  | <b>Objectives</b>                          | <b>2</b> |
| <b>3</b>  | <b>Problem Statement</b>                   | <b>2</b> |
| <b>4</b>  | <b>Circuit Analysis and Simplification</b> | <b>3</b> |
| 4.1       | Step-by-Step Derivation . . . . .          | 3        |
| <b>5</b>  | <b>Truth Table Verification</b>            | <b>4</b> |
| <b>6</b>  | <b>Hardware Implementation</b>             | <b>4</b> |
| 6.1       | Components Required . . . . .              | 4        |
| 6.2       | Pin Assignment . . . . .                   | 5        |
| 6.3       | Circuit Schematic . . . . .                | 5        |
| 6.4       | Working Principle . . . . .                | 5        |
| <b>7</b>  | <b>Arduino Program</b>                     | <b>6</b> |
| <b>8</b>  | <b>Assembly Program</b>                    | <b>7</b> |
| <b>9</b>  | <b>Results</b>                             | <b>8</b> |
| <b>10</b> | <b>Optimisation Analysis</b>               | <b>8</b> |
| 10.1      | Gate Count Comparison . . . . .            | 8        |
| 10.2      | Advantages of Simplification . . . . .     | 8        |
| <b>11</b> | <b>Conclusion</b>                          | <b>9</b> |

## 1. ABSTRACT

This report presents a systematic **analysis, simplification, and hardware implementation** of a digital logic circuit drawn from a GATE Electronics and Communication Engineering (EC) examination paper. Boolean algebra and DeMorgan's theorem are applied methodically to reduce the circuit, yielding the result:

$$Y = 1$$

An **Arduino UNO** platform is subsequently employed to verify this result experimentally. Three LEDs serve as indicators for the input variables  $A$ ,  $B$ , and  $C$ , while a fourth LED represents the output  $Y$ . The firmware automatically iterates through all eight possible input combinations and demonstrates that the output remains HIGH in every case, thereby confirming the theoretical derivation.

## 2. OBJECTIVES

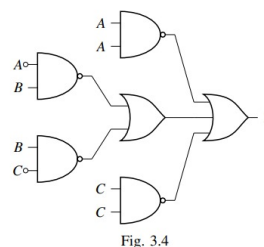
The following objectives guided the course of this project:

1. Analyse the structure of the given GATE EC logic circuit.
2. Derive the corresponding Boolean expression from the schematic.
3. Simplify the expression using Boolean algebra and DeMorgan's theorem.
4. Construct a complete truth table to verify the simplified result.
5. Implement the logic on an Arduino UNO platform and observe output behaviour.
6. Demonstrate the practical advantages of Boolean simplification and logic optimisation.

## 3. PROBLEM STATEMENT

Obtain the Boolean expression corresponding to the given logic circuit and simplify it using Boolean algebra. Implement the simplified result on an Arduino UNO and verify the output experimentally.

11. Obtain the Boolean Expression for the logic circuit shown below in Fig. 3.4.  
(GATE EC 1993)



**Figure 1** Given GATE EC Logic Circuit

## 4. CIRCUIT ANALYSIS AND SIMPLIFICATION

### 4.1 Step-by-Step Derivation

#### Step 1 — First-Stage NAND Gate Outputs

The outputs of the two input-stage NAND gates are expressed as:

$$X_1 = \overline{A} \overline{B} \quad X_2 = \overline{B} \overline{C}$$

#### Step 2 — Application of DeMorgan's Theorem

Applying DeMorgan's theorem to each expression:

$$X_1 = A + \bar{B} \quad X_2 = \bar{B} + C$$

#### Step 3 — OR Gate Combination

The OR gate combines  $X_1$  and  $X_2$ :

$$X_3 = X_1 + X_2 = A + \bar{B} + C$$

#### Step 4 — Self-Input NAND (NOT) Gates

Two additional NAND gates with identical inputs function as inverters:

$$X_4 = \overline{A \cdot A} = \bar{A} \quad X_5 = \overline{C \cdot C} = \bar{C}$$

#### Step 5 — Final Output Expression

$$\begin{aligned} Y &= X_3 + X_4 + X_5 \\ &= (A + \bar{B} + C) + \bar{A} + \bar{C} \\ &= (A + \bar{A}) + \bar{B} + (C + \bar{C}) \\ &= 1 + \bar{B} + 1 \end{aligned}$$

#### ✓ Verified Result

$$Y = 1$$

The output is **always logic HIGH**, irrespective of the values of inputs  $A$ ,  $B$ , or  $C$ . This establishes that the original circuit is logically redundant and may be replaced entirely by a direct connection to the supply voltage ( $V_{CC}$ ).

## 5. TRUTH TABLE VERIFICATION

| A | B | C | Output Y |
|---|---|---|----------|
| 0 | 0 | 0 | 1 ✓      |
| 0 | 0 | 1 | 1 ✓      |
| 0 | 1 | 0 | 1 ✓      |
| 0 | 1 | 1 | 1 ✓      |
| 1 | 0 | 0 | 1 ✓      |
| 1 | 0 | 1 | 1 ✓      |
| 1 | 1 | 0 | 1 ✓      |
| 1 | 1 | 1 | 1 ✓      |

**Table 1** Complete Truth Table — Output is always logic HIGH

The truth table confirms that  $Y = 1$  holds for all  $2^3 = 8$  possible combinations of the three input variables, thereby validating the Boolean simplification analytically.

## 6. HARDWARE IMPLEMENTATION

### 6.1 Components Required

| Component                               | Quantity    |
|-----------------------------------------|-------------|
| Arduino UNO Microcontroller Board       | 1           |
| Light Emitting Diodes (LEDs)            | 4           |
| 220 $\Omega$ Current-Limiting Resistors | 4           |
| Breadboard                              | 1           |
| Jumper Wires                            | As Required |

**Table 2** Bill of Materials

## 6.2 Pin Assignment

| Arduino Pin | Function                           |
|-------------|------------------------------------|
| D2          | Input Variable $A$ — LED Indicator |
| D3          | Input Variable $B$ — LED Indicator |
| D4          | Input Variable $C$ — LED Indicator |
| D5          | Output $Y$ — LED Indicator         |

Table 3 *Arduino UNO Pin Assignment*

## 6.3 Circuit Schematic

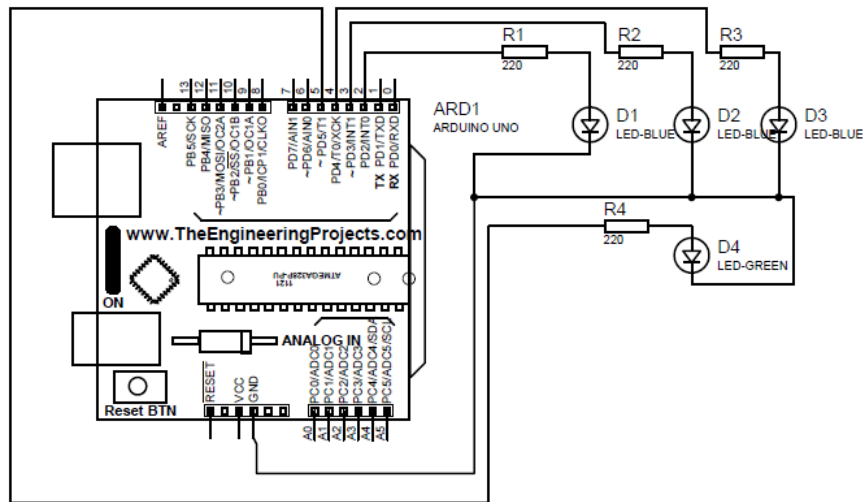


Figure 2 *Arduino UNO Implementation Schematic*

## 6.4 Working Principle

The Arduino firmware automatically generates all eight possible binary combinations of inputs  $A$ ,  $B$ , and  $C$  in a continuous loop. The LEDs connected to pins D2, D3, and D4 provide a visual indication of the current input state. The LED connected to pin D5 represents the output  $Y$ . Since the simplified Boolean expression evaluates to  $Y = 1$  unconditionally, the output LED remains **continuously illuminated** for every input combination, providing clear empirical confirmation of the simplification.

## 7. ARDUINO PROGRAM

---

```
1  const int A_LED = 2;    // Input A indicator
2  const int B_LED = 3;    // Input B indicator
3  const int C_LED = 4;    // Input C indicator
4  const int Y_LED = 5;    // Output Y (always HIGH)
5
6  void setup()
7  {
8      pinMode(A_LED, OUTPUT);
9      pinMode(B_LED, OUTPUT);
10     pinMode(C_LED, OUTPUT);
11     pinMode(Y_LED, OUTPUT);
12 }
13
14 void loop()
15 {
16     for (int i = 0; i < 8; i++)
17     {
18         int A = (i >> 2) & 1; // Extract bit 2
19         int B = (i >> 1) & 1; // Extract bit 1
20         int C = i          & 1; // Extract bit 0
21
22         digitalWrite(A_LED, A);
23         digitalWrite(B_LED, B);
24         digitalWrite(C_LED, C);
25
26         // Y = 1 always; output LED remains ON
27         digitalWrite(Y_LED, HIGH);
28
29         delay(1000); // Hold each combination for 1
                       // second
30     }
31 }
```

Full source code on GitHub:

[github.com/samikshas1118/IITH-SRFP/blob/main/Arduino.Code](https://github.com/samikshas1118/IITH-SRFP/blob/main/Arduino.Code)

## 8. ASSEMBLY PROGRAM

---

```
1  .include "m328pdef.inc"      ; ATmega328P register
   definitions
2
3  .org 0x0000
4  rjmp MAIN                    ; Reset vector -> jump to MAIN
5
6  MAIN:
7      ldi r16, 0b00111100      ; Configure DDRD: pins 2-5 as
   outputs
8      out DDRD, r16
9
10 LOOP:
11     ldi r17, 0                ; Initialise loop counter to 0
12
13 NEXT:
14     mov r18, r17
15     lsl r18                    ; Shift left twice to align
   with PORTD pins 2-4
16     lsl r18
17     out PORTD, r18            ; Drive A, B, C LEDs
18
19     sbi PORTD, 5              ; Set pin D5 HIGH (Y = 1 always
   )
20
21     rcall DELAY               ; Approximately 1-second delay
22
23     inc r17
24     cpi r17, 8
25     brne NEXT                ; Repeat for all 8 input
   combinations
26
27     rjmp LOOP                 ; Restart cycle indefinitely
28
29
30 DELAY:
31     ldi r20, 100
32 L1: ldi r21, 255
33 L2: ldi r22, 255
34 L3: dec r22
35     brne L3
36     dec r21
37     brne L2
38     dec r20
39     brne L1
40     ret
```



Full source code on GitHub:

[github.com/samikshas1118/IITH-SRFP/blob/main/Assembly\\_Code](https://github.com/samikshas1118/IITH-SRFP/blob/main/Assembly_Code)

## 9. RESULTS

### ♣ Experimental Findings

The Arduino UNO successfully generated all eight possible input combinations of  $A$ ,  $B$ , and  $C$  in sequence. In every case, the output LED connected to pin D5 remained **continuously illuminated**, experimentally confirming:

$$Y = 1$$

Both the C++ (Arduino) implementation and the AVR Assembly implementation produced identical results, demonstrating consistency across both levels of abstraction.

## 10. OPTIMISATION ANALYSIS

### 10.1 Gate Count Comparison

| Metric            | Original Circuit | Simplified          |
|-------------------|------------------|---------------------|
| NAND Gates        | 4                | 0                   |
| OR Gates          | 2                | 0                   |
| Total Gates       | 6                | 0                   |
| Propagation Delay | Multi-stage      | None                |
| Implementation    | Complex logic    | V <sub>CC</sub> tie |

**Table 4** *Original Circuit vs. Simplified Implementation*

### 10.2 Advantages of Simplification

- **Complete elimination of hardware** — all six logic gates are removed, replaced by a single connection to V<sub>CC</sub>.
- **Zero propagation delay** — no gate switching is involved.
- **Reduced power consumption** — no active logic components draw quiescent current.
- **Improved reliability** — fewer components result in fewer potential failure points.

- **Smaller PCB footprint** — significant area savings in integrated circuit layouts and printed circuit board designs.

## 11. CONCLUSION

---

The given GATE EC logic circuit was systematically **analysed and simplified** using Boolean algebra and DeMorgan's theorem. The final Boolean expression was found to evaluate to  $Y = 1$ , establishing that the circuit output remains at logic HIGH for every possible combination of inputs, rendering the underlying gate network functionally redundant.

An **Arduino UNO** based implementation was subsequently developed and tested. The experimental observations were in complete agreement with the theoretical analysis. This project underscores the practical significance of **Boolean simplification** and **logic optimisation** in reducing hardware cost, minimising power consumption, and streamlining design complexity in digital electronic systems.

— End of Report —